SZOFTVERFEJLESZTŐ- ÉS TESZTELŐ TECHNIKUS SZAK

Luxory Products Collection Webshop

LARAVEL projekt

Csapat név:

Professorado

Csapat tagok:

Dr. Bölkény Ildi

Dr. Forgács Zsófia

Miskolc

2026

1. Tartalom

2. Bevezetés 3
3. Laravel PHP projektek 3
4. Mi a Laravel és miért előnyös? 4
5. A Laravel projekt általános felépítése 5
6. MVC szemlélet és a működés logikája 8
7. 5. A felhasznált technológiák külön bemutatása 9
7.1. HTML 9
7.2. PHP 9
7.3. CSS 10
7.4. MySQL 10
7.5. Laravel 10
7.6. Blade 10
8. A webshop jellegű Laravel projektünk működése 11
9. Adatbázis és CRUD műveletek 13
10. Biztonság, validáció és hitelesítés 13
11. A rendszer működésének bemutatása 14
12. A rendszer technikai működése és kódszintű háttere 21
12.1. Az útvonalkezelés szerepe 21
12.2. A controller mint vezérlőközpont 22
12.3. A modell és az adatbázis kapcsolata 23
12.4. Az új termék létrehozásának folyamata 24
12.5. Validáció és mentés a controllerben 25
12.6. A képek speciális kezelése 26
12.7. A listaoldal dinamikus működése 27
12.8. A részletező oldal és az adatmódosítás 28
12.9. Bejelentkezés, felhasználókezelés és menüsor 30
12.10. Szakmai megjegyzések és továbbfejlesztési pontok 31
12.11. Összefoglaló technikai értelmezés 32
13. Továbbfejlesztési lehetőségek 33
14. Tesztelés, dokumentálás és beadási szempontok 34
15. Összegzés 35
16. Felhasznált irodalom 36

2. Bevezetés

A jelen vizsgafeladat keretében egy webáruház jellegű Laravel-alapú webalkalmazás készült, amely egy egyszerűen kezelhető, mégis jól strukturált termékkezelő rendszert valósít meg. A fejlesztés célja egy olyan online felület létrehozása volt, ahol a különböző termékek áttekinthető formában jeleníthetők meg, valamint az adatok kezelése is hatékonyan megoldható.

1. ábra: Kliens – Vizsgafeladat

Az elkészült rendszer lehetőséget biztosít új termékek feltöltésére, a meglévő elemek módosítására, törlésére és listázására. A termékekhez név, szöveges leírás, ár és kép is megadható, így a felhasználó részletes és vizuálisan is jól értelmezhető formában kezelheti az egyes elemeket. Ez a megoldás jól szemlélteti, hogyan alkalmazható a Laravel keretrendszer egy gyakorlati feladat során egy olyan projektben, ahol a funkcionalitás, az áttekinthetőség és a bővíthetőség egyaránt fontos szerepet kap.

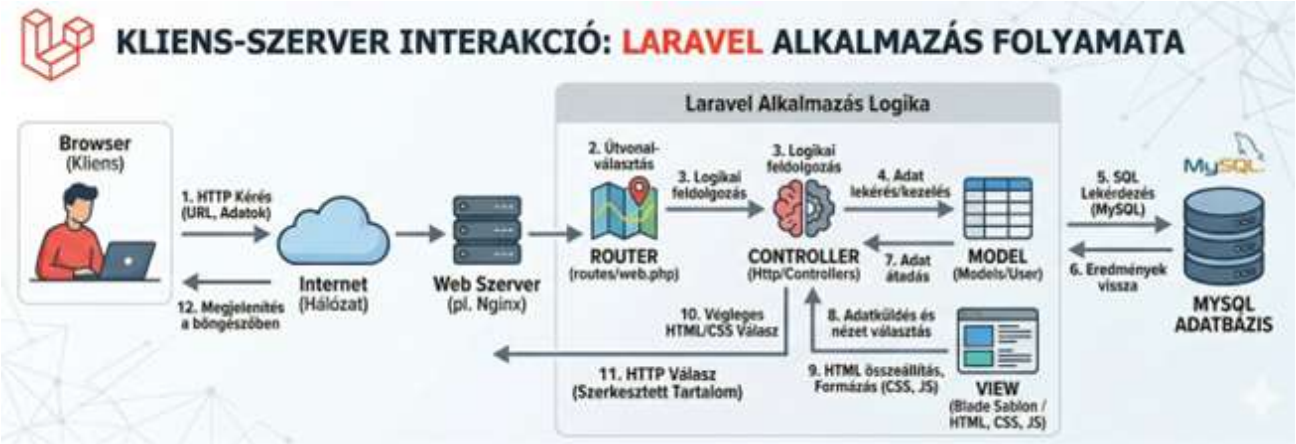
3. Laravel PHP projektek

A mai webfejlesztésben fontos követelmény, hogy az elkészült alkalmazás ne csak működőképes legyen, hanem áttekinthető, bővíthető és karbantartható

is. Amíg egy egyszerű statikus oldal néhány HTML fájllal is elkészíthető, addig egy adatbázis-kapcsolt, űrlapokat kezelő vagy adminfelülettel rendelkező rendszer esetében már célszerű keretrendszert használni.

A Laravel egy modern PHP keretrendszer, amely ilyen projektekhez szabványos és jól szervezett megoldásokat biztosít. Segítségével a fejlesztő nem véletlenszerűen helyezi el a kódot, hanem jól elkülönített részekre bontja a programot, így a projekt logikája később is könnyen követhető marad.

Jelen dokumentum célja, hogy szakmailag rendezett, de könnyen érthető bemutatást adjon a Laravel projektek felépítéséről, valamint a hozzájuk tartozó alaptechnológiákról, külön kiemelve a HTML, PHP, CSS és MySQL szerepét.



2. ábra: Kliens – szerver folyamatábra

4. Mi a Laravel és miért előnyös?

A Laravel nyílt forráskódú PHP keretrendszer, amely megkönnyíti a modern webalkalmazások fejlesztését. Számos előre elkészített megoldást biztosít, például útvonalkezelést, hitelesítést, validációt, adatbázis-kezelést és sablonmotor-támogatást.

A keretrendszer egyik legfontosabb előnye a rendezett projektstruktúra. A fejlesztő külön kezeli a modelleket, a kontrollereket és a nézeteket, így a rendszer logikája átláthatóbbá válik, a hibák pedig gyorsabban megtalálhatók.

További előnyt jelent az Eloquent ORM használata, amely objektumorientált módon egyszerűsíti az adatbázis műveleteket, valamint a Blade sablonmotor, amely tiszta és jól olvasható nézetkódot tesz lehetővé.

1. táblázat: A Laravel fő előnyei röviden

Előny	Jelentése a gyakorlatban
MVC szerkezet	A kód logikusan tagolt, ezért könnyebb javítani és bővíteni.
Blade sablonok	A nézetek dinamikusan épülnek fel, mégis áttekinthetők maradnak.
Eloquent ORM	Az adatbázis-műveletek egyszerűbben kezelhetők.
Validáció	A hibás adatok kiszűrhetők a mentés előtt.
Middleware	A hozzáférések és kérések szabályozhatók.

5. A Laravel projekt általános felépítése

Egy Laravel projekt mappaszerkezete előre meghatározott, ami megkönnyíti a fejlesztést és a dokumentálást. Az alkalmazás üzleti logikája jellemzően az app mappában található, az útvonalak a routes könyvtárban, a megjelenítésért felelős nézetek pedig a resources/views mappában helyezkednek el.

Az adatbázissal kapcsolatos fájlok a database mappában találhatók. Ide tartoznak a migrációk, amelyek meghatározzák a táblák létrehozását és

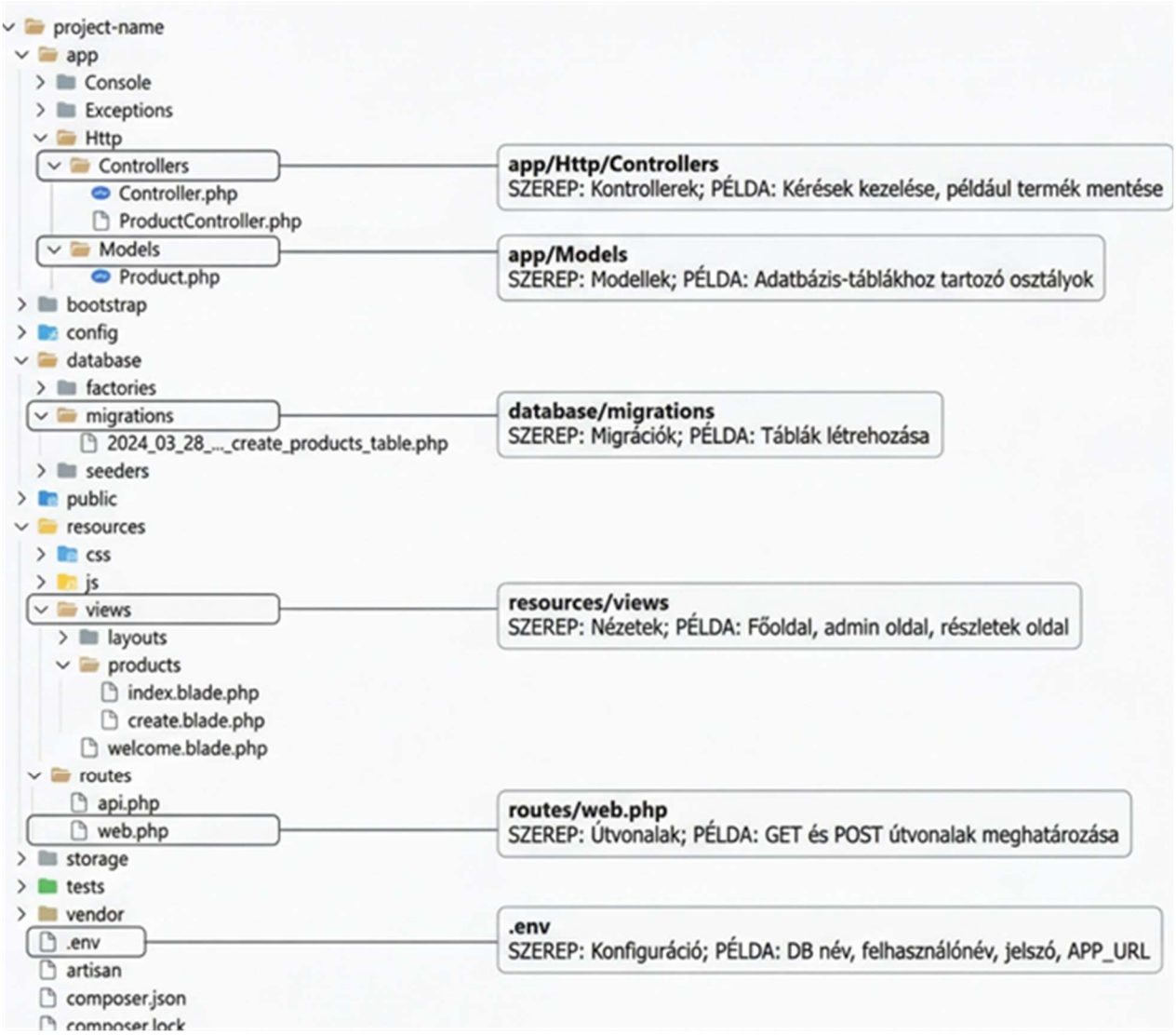
módosítását. A .env fájl a projekt környezeti beállításait tartalmazza, például az adatbázis nevét, felhasználóját és jelszavát.

A Laravel projekt felépítése jól átgondolt mappastruktúrán alapul, amely segíti a fejlesztőt abban, hogy a különböző feladatokat elkülönítve, rendezett formában kezelje. Az app/Http/Controllers mappa a kontrollereket tartalmazza, amelyek a beérkező kérések feldolgozásáért felelnek. Ezek kezelik például a termékek mentését, törlését, frissítését, valamint azt is, hogy melyik nézet jelenjen meg a felhasználó számára.

Az app/Models könyvtárban találhatók a modellek, amelyek az adatbázis egyes tábláihoz kapcsolódó osztályokat jelentik. Ezek biztosítják az adatbázissal való kapcsolatot, valamint az adatok lekérdezését, tárolását és módosítását. Egy webshop esetében ilyen lehet például a Product modell, amely a termékek adatait kezeli.

A resources/views mappa a nézeteket tartalmazza, vagyis azokat a felületeket, amelyeket a felhasználó a böngészőben lát. Ide tartozik például a főoldal, az adminfelület, a termékek listázását végző oldal vagy a részletes terméknézet. Ezek a fájlok felelnek az alkalmazás vizuális megjelenítéséért.

A routes/web.php fájlban kerülnek meghatározásra az alkalmazás útvonalai. Ez a fájl dönti el, hogy egy adott URL-cím vagy kérés melyik controller metódushoz kerüljön. Itt adjuk meg például a GET és POST típusú útvonalakat, amelyek a felhasználói navigációt és az űrlapkezelést teszik lehetővé.



3. ábra: Projekt struktúra

A database/migrations mappa a migrációs fájlokat tárolja. Ezek segítségével hozhatók létre vagy módosíthatók az adatbázis táblái szabályozott, követhető módon. A migrációk lehetővé teszik, hogy az adatbázis szerkezete a projekt részeként kezelhető legyen, így a fejlesztés és a telepítés is egyszerűbbé válik.

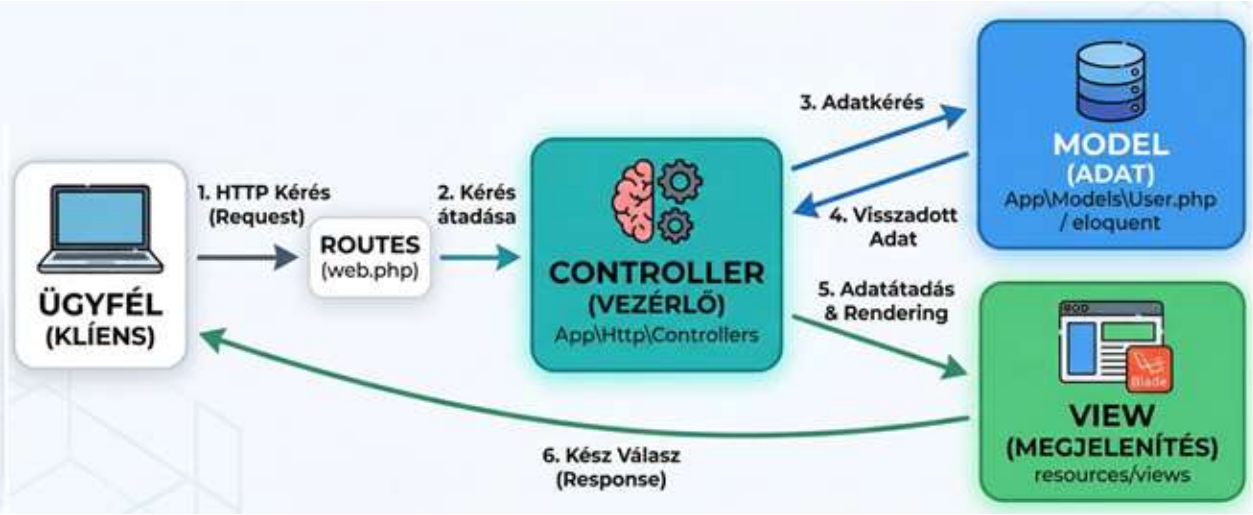
A .env fájl az alkalmazás környezeti beállításait tartalmazza. Itt adhatók meg például az adatbázis neve, a felhasználónév, a jelszó, valamint az alkalmazás alapértelmezett elérési útja (APP_URL). Ez a fájl fontos

szerepet tölt be abban, hogy a projekt különböző környezetekben – például helyi gépen vagy szerveren – is megfelelően működjön.

6. MVC szemlélet és a működés logikája

A Laravel az MVC, vagyis a Model–View–Controller elvet követi. A Model felel az adatok kezeléséért, a View a megjelenítésért, a Controller pedig a beérkező kérések feldolgozásáért és a megfelelő válasz összeállításáért.

Egy webshop esetében ez úgy működik, hogy a felhasználó megnyit egy oldalt vagy kitölt egy űrlapot. Az útvonal a megfelelő controller metódushoz irányítja a kérést, a controller pedig elvégzi a validációt, adatbázis-műveletet vagy egyéb logikát, majd egy nézetet ad vissza.



4. ábra: Egyszerü LARAVEL MVC felépités – az adatáramlás folyamata

Az ábra a Laravel keretrendszer MVC felépítését és az adatok áramlásának folyamatát szemlélteti. A működés a kliensoldalról indul, amikor a felhasználó egy HTTP-kérést küld az alkalmazás felé. Ezt a kérést a route-rendszer (web.php) fogadja, majd továbbítja a megfelelő controllernek. A controller feladata a kérés feldolgozása, az üzleti logika kezelése, valamint a szükséges adatok lekérése a modelből. A model biztosít kapcsolatot az adatbázissal, így innen történik az adatok olvasása,

módosítása vagy mentése. Miután a controller megkapta a szükséges adatokat, átadja azokat a view rétegnek, amely a felhasználó számára megjeleníthető formába rendezi az információt. Végül a rendszer egy kész választ küld vissza a kliensnek. Az MVC struktúra előnye, hogy jól elkülöníti egymástól az adatkezelést, a vezérlési logikát és a megjelenítést, ezáltal a program átláthatóbbá, karbantarthatóbbá és könnyebben bővíthetővé válik.

2. táblázat: Az egyes elemek feladata

Elem	Feladat	Webshop példa
Model	Adatkezelés, mentés, lekérdezés	Product modell
View	Felhasználói felület megjelenítése	terméklista, részletek oldal
Controller	Kérések fogadása és feldolgozása	mentés, törlés, frissítés

7. 5. A felhasznált technológiák külön bemutatása

7.1. HTML

A HTML a weboldal szerkezetét írja le. Meghatározza a címsorokat, bekezdéseket, képeket, űrlapokat, táblázatokat és gombokat. Egy webshop projektben HTML segítségével épül fel a kezdőlap, a terméklista és az admin felület űrlaprendszere.

7.2. PHP

A PHP a szerveroldali programozási nyelv, amely feldolgozza a felhasználói kéréseket, kapcsolatot tart az adatbázissal, és dinamikus tartalmat készít elő a böngésző számára. A Laravel maga is PHP-alapú keretrendszer.

7.3. CSS

A CSS a megjelenésért felel. Segítségével állítható be az oldalak színvilága, betűstílusa, térközei, gombjainak kialakítása és az általános vizuális elrendezés.

7.4. MySQL

A MySQL relációs adatbázis-kezelő rendszer, amely a projekt adatait tárolja. Ide kerülnek a termékek nevei, leírásai, árai és képei, valamint adott esetben a felhasználói adatok is.

7.5. Laravel

A Laravel összefogja és rendszerezi a teljes alkalmazást. Feladata az útvonalkezelés, a sablonok kezelése, a middleware-ek használata, a hitelesítés, az adatkezelés és a validáció támogatása.

7.6. Blade

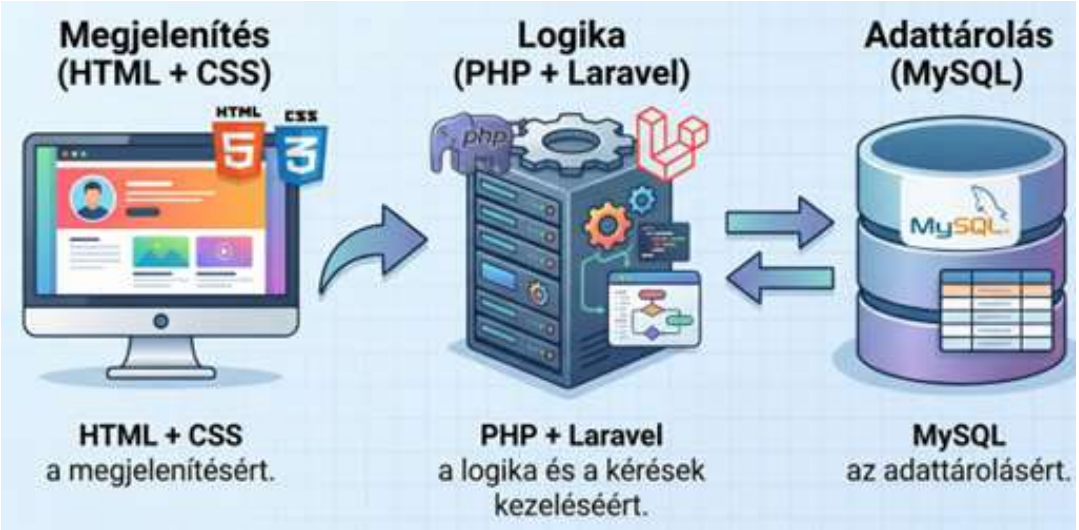
A Blade a Laravel saját sablonmotorja, amely lehetővé teszi, hogy a HTML kódba tisztán és áttekinthetően kerüljön be a dinamikus adat.

3. táblázat: Alkalmazott technológiák összefoglalása

Technológia	Feladata	Példa a projektben
HTML	Oldalszerkezet	Űrlapok, címek, termékkártyák
PHP	Szerveroldali logika	Mentés, törlés, adatfeldolgozás
CSS	Megjelenés és stílus	Színek, gombok, elrendezés
MySQL	Adattárolás	Termékek és felhasználók
Laravel	Keretrendszer és	Route, controller,

	szabályozás	middleware
Blade	Dinamikus nézetek	Terméklista megjelenítése adatbázisból

Az ábra szemléletesen mutatja be a webalkalmazás három fő technológiai rétegét. A megjelenítésért a HTML és a CSS felel, amelyek biztosítják az oldal szerkezetét és vizuális kialakítását. A középső rétegben a PHP és a Laravel végzi az alkalmazás logikai működését, vagyis a kérések feldolgozását és az üzleti folyamatok kezelését. Az adatok tárolását és visszakeresését a MySQL adatbázis-kezelő rendszer biztosítja. A három elem együttműködése teszi lehetővé egy dinamikus, adatbázis-alapú webshop működését.



5. ábra: Hogyan müködik?

8. A webshop jellegű Laravel projektünk működése

A jelen projekt egy Laravel alapú webshop jellegű webalkalmazás, amelynek fő célja a termékek kezelése és megjelenítése. A felhasználó a böngészőben megnyitja a kezdőoldalt, ahol a rendszer lekéri az adatbázisban tárolt termékeket, majd listázza azokat a felületen. A termékek kártyás elrendezésben jelennek meg, így a látogató könnyen áttekintheti a

kínálatot, majd egy részletes oldal segítségével bővebb információt is megtekinthet az adott elemről. A terméklista megjelenítését az index() metódus végzi, amely a termékeket időrendben lekéri és átadja a nézetnek

A rendszer adminisztrációs felülettel is rendelkezik, ahol új termék rögzíthető. A termékfeltöltő oldalon a felhasználó megadhatja a termék nevét, leírását, árát és képét. Az adatok beküldése után a rendszer ellenőrzi az űrlap helyességét, majd megfelelő kitöltés esetén elmenti a rekordot az adatbázisba. A feltöltött képet a program feldolgozza, JPEG formátumban kódolja, majd a termék többi adatával együtt tárolja el

A webshop működésének fontos része, hogy az egyes termékek külön részletező oldalon is megjeleníthetők. Itt a kiválasztott termék képe, neve, ára és leírása látható. A jelenlegi megoldás lehetőséget ad a termék törlésére, valamint a leírás módosítására is, így a rendszer részben a CRUD műveletek teljes körét lefedi. A részletes nézetet a show_details() metódus tölti be, a törlést a destroy(), míg a leírás frissítését az updateDescription() végzi

4. táblázat: Müködés

Lépés	Felhasználói művelet	Rendszer művelete	Eredmény
1	Oldal megnyitása	Controller lekéri a termékeket	Megjelenik a terméklista
2	Új termék felvitele	Validáció és mentés	Új rekord jön létre
3	Termék részletek megtekintése	A kiválasztott rekord lekérdezése	Részletező oldal jelenik meg
4	Törlés vagy módosítás	Adatbázis-rekord frissítése vagy törlése	A lista frissül

9. Adatbázis és CRUD műveletek

A jelen projektben a Create művelet a store() metóduson keresztül valósul meg, ahol a rendszer új terméket hoz létre és elmenti annak nevét, leírását, árát és képét. A Read művelet az index() és a show_details() metódusokban jelenik meg, mivel ezek felelnek a termékek listázásáért és a részletes termékoldal megjelenítéséért. A Delete művelet a destroy() metódushoz kapcsolódik, amely eltávolítja a kiválasztott terméket az adatbázisból. Az Update jelenleg részlegesen van megvalósítva, mivel az általános edit() és update() metódusok még nem tartalmaznak működő logikát, azonban a termékleírás módosítása külön funkcióként elérhető az updateDescription() metódus segítségével

Ez jól mutatja, hogy a projekt már most is egy működő adatbázis-alapú rendszer, ugyanakkor továbbfejleszthető lenne teljes körű szerkesztési lehetőségekkel is. A Laravel előnye ebben az esetben az, hogy a controller, a modell és a nézet jól elkülönül egymástól, így a későbbi bővítés egyszerűbben megoldható.

10. Biztonság, validáció és hitelesítés

A professzionális webalkalmazások egyik legfontosabb tulajdonsága a biztonság. Nem elegendő csupán adatot fogadni és eltárolni, hanem ellenőrizni kell azt is, hogy a beküldött információ megfelelő formátumú-e, nem hiányos-e, valamint hogy a felhasználó rendelkezik-e jogosultsággal az adott művelet végrehajtására.

A Laravel ebben nagy segítséget nyújt, mivel beépített validációs lehetőségeket biztosít. A projektben a termék mentése előtt a rendszer ellenőrzi, hogy a termék neve kötelező mező legyen, szöveges típusú adatot tartalmazzon, a leírás szintén kötelező legyen, az ár numerikus formátumú legyen, a feltöltött fájl pedig valódi képként legyen értelmezhető. Ez megakadályozza, hogy hibás vagy hiányos adatok kerüljenek az adatbázisba

A hitelesítés szintén fontos szerepet kap a Laravel projektekben. A feltöltött HomeController alapján a rendszer middleware használatával biztosítja, hogy bizonyos oldalak csak bejelentkezett felhasználók számára legyenek elérhetők. Ez azt jelenti, hogy az alkalmazás képes megkülönböztetni az azonosított és a nem azonosított felhasználókat, ami különösen fontos adminisztrációs funkciók esetén

A fájlkezelés szintén a biztonság részét képezi, különösen webshop esetében, ahol képfeltöltés is történik. A rendszer csak megfelelő képfájlt fogad el, majd azt feldolgozza és a termékhez kapcsolva eltárolja. Ez csökkenti a hibás vagy nem támogatott fájlformátumokból eredő problémák lehetőségét

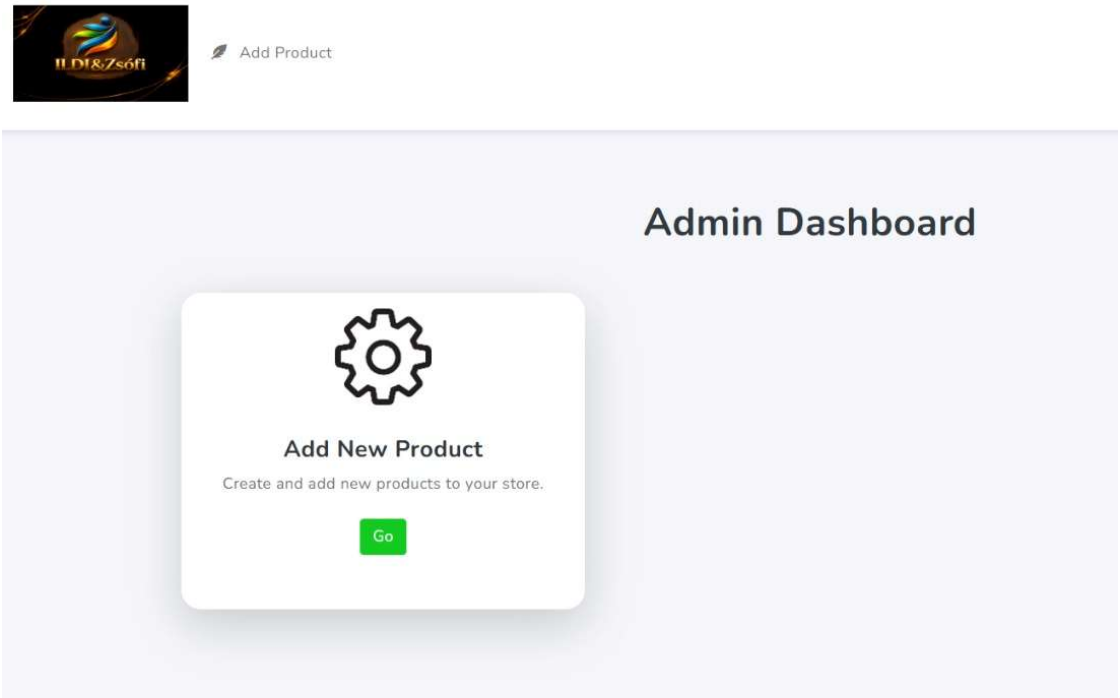
5. táblázat: Biztonsági és működési szempontok

Szempont	Jelentés
Validáció	A hibás vagy hiányos adatok kiszűrése mentés előtt.
Hitelesítés	Annak ellenőrzése, hogy a felhasználó be van-e jelentkezve.
Jogosultság	Bizonyos műveletek csak admin vagy engedélyezett felhasználó számára érhetők el.
Fájlkezelés	A feltöltött képek típusának és méretének ellenőrzése.

11. A rendszer működésének bemutatása

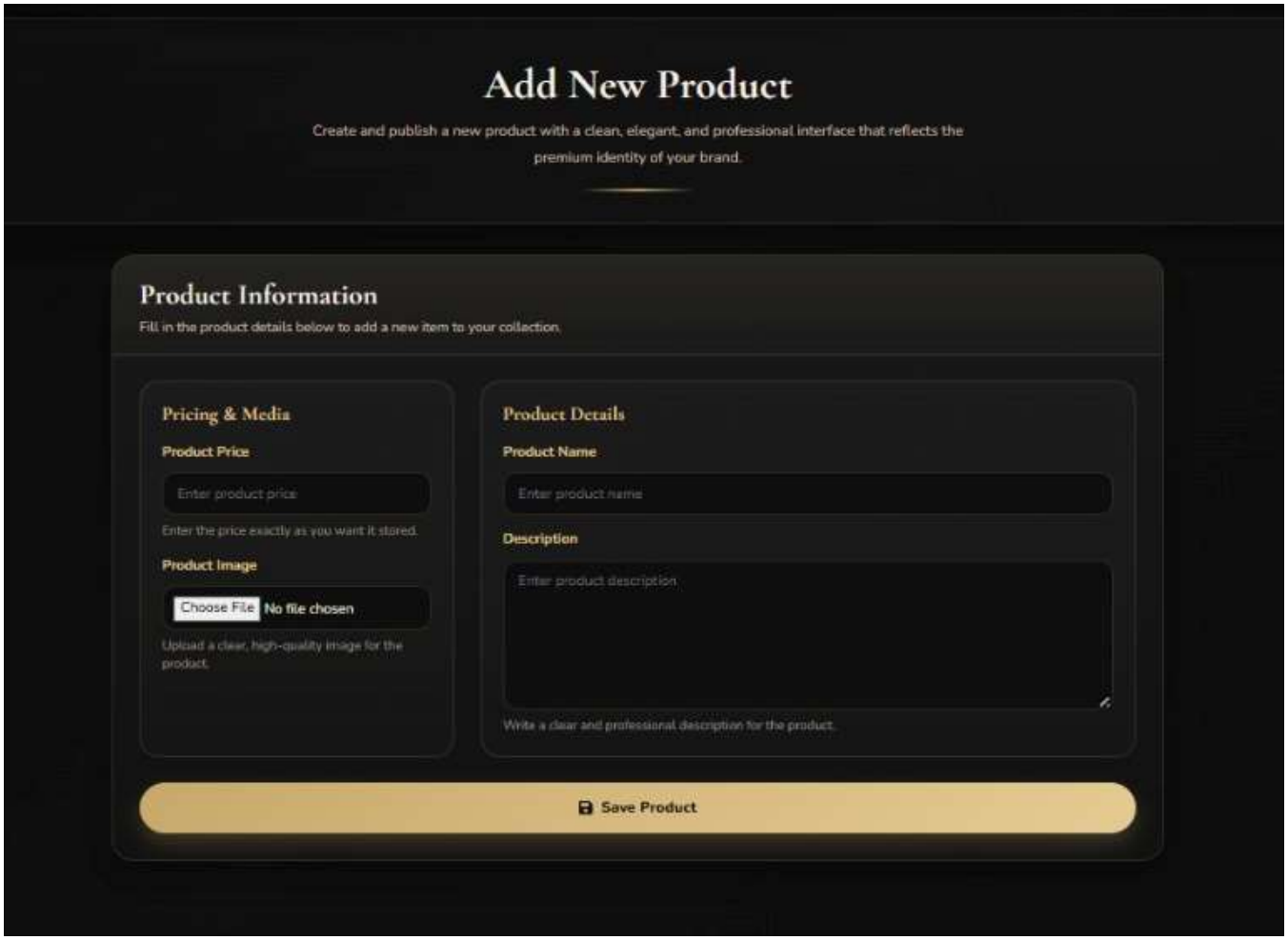
A fejlesztett webalkalmazás egy Laravel alapú webshop, amely lehetőséget biztosít a termékek létrehozására, megjelenítésére, részletes megtekintésére, valamint bizonyos adatok módosítására és törlésére. A rendszer működése jól elkülöníthető adminisztrációs és felhasználói felületekre épül. Az adminisztrációs rész a termékek kezelését támogatja,

míg a publikus felület a látogatók számára jeleníti meg az adatbázisban tárolt elemeket.



6. ábra: Az adminisztrációs kezdőfelület

A működés első lépése az adminisztrációs felület elérése. Az admin dashboard egy egyszerű és áttekinthető kezdőoldal, amelynek célja, hogy az adminisztrátor gyorsan hozzáférjen a legfontosabb kezelési funkciókhoz. A képen látható felületen közvetlenül elérhető az új termék hozzáadására szolgáló lehetőség. Ez a megoldás azért előnyös, mert a rendszer kezelése egyértelmű, és a termékfeltöltési folyamat könnyen elindítható.

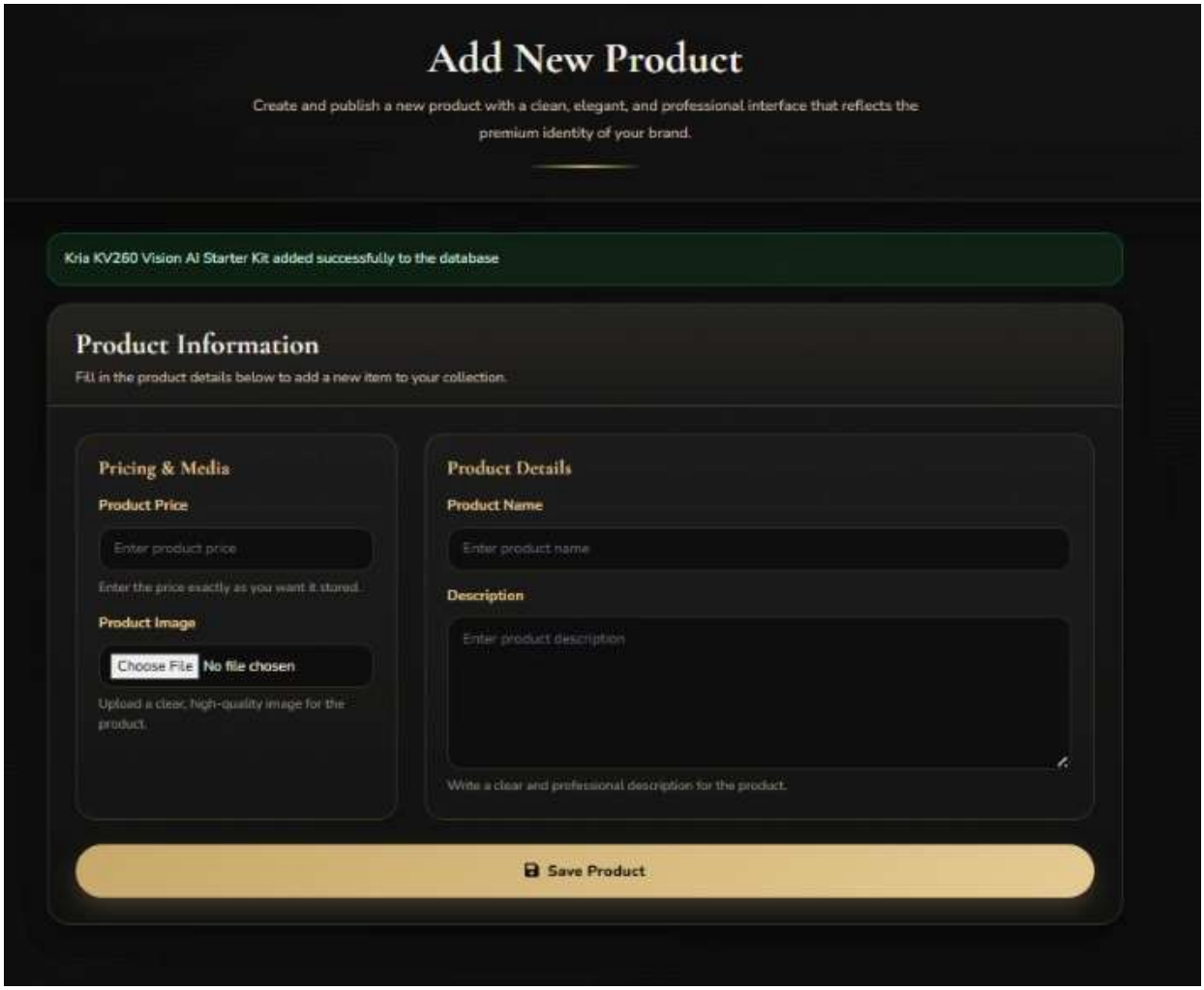


7. ábra: Új termék hozzáadására szolgáló űrlap

Az adminisztrációs oldalról a felhasználó az új termék létrehozására szolgáló űrlapra jut. Ezen a felületen megadható a termék neve, leírása, ára, valamint lehetőség van kép feltöltésére is. Az űrlap felépítése logikus és jól tagolt, így a felhasználó könnyen kitöltheti a szükséges adatokat. A rendszer ezzel a felülettel valósítja meg a Create műveletet, vagyis az új adatbázis-rekord létrehozását.

A mentés során a rendszer ellenőrzi a beküldött adatokat. A termék neve és leírása kötelező mező, az árnak számként kell szerepelnie, a feltöltött fájlnak pedig valódi képfájlnak kell lennie. Ez a validáció biztosítja, hogy csak megfelelő formátumú és hiánytalan adatok kerüljenek be az adatbázisba. A képfeltöltés különösen fontos webshop esetén, mivel a

termékek vizuális megjelenítése jelentős szerepet játszik a felhasználói élményben.



8. ábra: Sikeres termékmentés visszajelzése

Sikeres mentés után a rendszer visszajelző üzenetet jelenít meg, amely tájékoztatja a felhasználót arról, hogy az új termék rögzítése megtörtént. Ez a visszacsatolás fontos része a felhasználóbarát működésnek, mivel egyértelműen jelzi a művelet eredményét. A mentés után a termék adatai már az adatbázisban szerepelnek, így megjeleníthetők a webshop nyilvános felületén is.

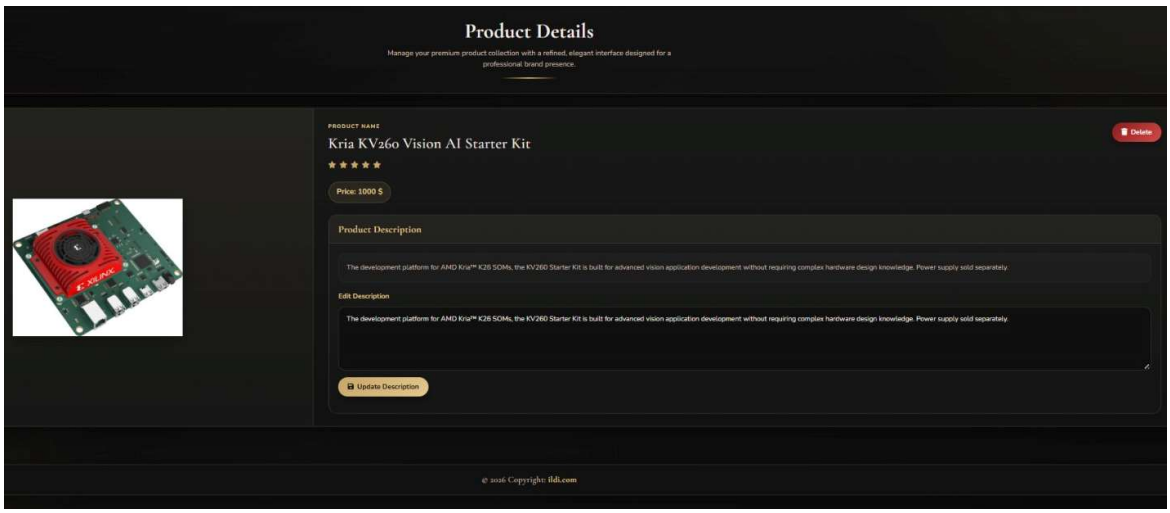
9. ábra: A webshop főoldala és a terméklista megjelenése

A következő lépésben a rendszer a főoldalon listázza az adatbázisban tárolt termékeket. A kezdőoldal feladata, hogy a látogatók számára áttekinthető formában bemutassa az elérhető elemeket. A képen látható termékkártya tartalmazza a termék képét, nevét, valamint egy részletező oldalra vezető gombot. A felület sötét, elegáns megjelenése és a kártyás elrendezés professzionális webshop benyomását kelti, miközben a felhasználó számára egyszerű böngészést tesz lehetővé.

Egy alap webshop projekt a beadás szempontjából megfelelő lehet, azonban számos olyan funkció létezik, amellyel a rendszer professzionálisabbá tehető. Ilyen például a termékkategóriák bevezetése, a kereső fejlesztése, a kosár és rendeléskezelés kialakítása, valamint a részletes admin jogosultságok megvalósítása.

További lehetőség a képek külön fájlként történő tárolása, az adatbázis normalizálása, valamint a reszponzív felület fejlesztése. A projekt minőségét emeli, ha a fejlesztő dokumentálja a route-okat, a controller metódusokat és az adatbázis mezőit is.

Ez a rész a Read művelet gyakorlati megvalósítása, hiszen a rendszer lekéri az adatbázisban tárolt adatokat, majd megjeleníti azokat a felhasználó számára. A terméklista így nem statikus tartalomként jelenik meg, hanem dinamikusan, az adatbázisból betöltve.



10. ábra: A termék részletező oldala

A termékkártyán található gomb segítségével a felhasználó megnyithatja az adott elem részletes oldalát. Itt már nagyobb méretben látható a termék képe, valamint megjelenik a név, az ár és a leírás is. Ez a felület részletesebb információt ad az adott termékről, így a látogató jobban megismerheti annak tulajdonságait.

A részletező oldal ugyanakkor nemcsak megjelenítésre szolgál, hanem bizonyos adminisztratív műveleteket is támogat. A képernyőkép alapján lehetőség van a termék törlésére, valamint a leírás módosítására is. Ez azt jelenti, hogy a rendszer a Delete műveletet teljes egészében, az Update műveletet pedig részlegesen valósítja meg. A jelenlegi fejlesztési

állapotban tehát az új termék létrehozása, a termékek listázása, a részletek megtekintése, a törlés, valamint a leírás frissítése működik.

11. ábra: Menüsor

A webshop felső részén elhelyezett navigációs sáv a felhasználói tájékozódást és az egyes funkciók gyors elérését szolgálja. A menüsorban megjelenő elemek – például a Products, About Us, Contact Us, Language és Login menüpontok – azt a célt szolgálják, hogy a látogató könnyen átláthassa az oldal felépítését, és gyorsan eljusson a számára fontos tartalmakhoz. Egy jól kialakított menürendszer nemcsak esztétikai szempontból fontos, hanem jelentősen javítja a felhasználói élményt is.

A Products menüpont elsődleges feladata a termékek megjelenítéséhez és böngészéséhez kapcsolódik, vagyis ezen keresztül érheti el a felhasználó a webshop kínálatát. Az About Us oldal általában a vállalkozás, a projekt vagy az oldal céljának bemutatására szolgál, így bizalmat építhet a látogatók felé. A Contact Us menüpont a kapcsolatfelvétel lehetőségét biztosítja, ahol például elérhetőségek, üzenetküldési lehetőség vagy további információk jelenhetnek meg. A Language opció arra utal, hogy a rendszer alkalmas lehet többnyelvű működésre is, ami különösen hasznos lehet eltérő nyelvű felhasználók kiszolgálása esetén. A Login funkció pedig a hitelesítéshez kapcsolódik, vagyis lehetővé teheti, hogy a jogosult felhasználók vagy adminisztrátorok belépjenek a rendszer védett részeire.

Összességében a webshop működése jól szemlélteti egy Laravel alapú, adatbázis-kapcsolt webalkalmazás felépítését. A rendszer logikusan elkülönített felületeken keresztül valósítja meg a termékkezelést, miközben biztosítja az adatok ellenőrzését, tárolását és megjelenítését. Ez a megoldás jó példája annak, hogyan használható a Laravel keretrendszer egy gyakorlati vizsgafeladatban egy működő, bővíthető és átlátható webshop kialakítására.

12. A rendszer technikai működése és kódszintű háttere

A webshop működését legegyszerűbben úgy lehet megérteni, ha végigkövetjük, hogyan jut el egy felhasználói kérés a böngészőtől egészen az adatbázisig, majd onnan vissza a megjelenített weboldalig. Laravelben ezt a folyamatot több, egymásra épülő réteg valósítja meg: az **útvonalak (routes)** fogadják a kérést, a **controller** eldönti, mi történjen, a **model** kapcsolatot teremt az adatbázissal, a **view** pedig megjeleníti az eredményt a felhasználó számára. A projektben ez a működés jól nyomon követhető a web.php, a ProductController, a Product modell és a Blade nézetek együttműködésén keresztül.

12.1. Az útvonalkezelés szerepe

A Laravel alkalmazásban az első belépési pont általában a routes/web.php fájl. Itt van megadva, hogy egy adott URL milyen vezérlőmetódust hívjon meg. A te projektedben például a kezdőoldal, a részletező oldal, az új termék mentése és a törlés is külön route-okon keresztül működik.

```
Route::get('/', [App\Http\Controllers\ProductController::class,
'index'])->name('welcome');
Route::get('/show_details/{id}', [ProductController::class,
'show_details'])->name('show_details');
Route::post('/newproduct/store', [ProductController::class,
'store'])->name('newproduct.store');
```

```
Route::delete('/products/{id}', [ProductController::class,
'destroy'])->name('products.destroy');
```

Ez nagyon egyszerűen azt jelenti, hogy:

- a / cím megnyitásakor a ProductController@index fut le,
- a /show_details/azonosító megnyitásakor az adott termék részletes oldala töltődik be,
- a POST típusú newproduct/store kérés új terméket ment,
- a DELETE típusú products/{id} kérés törli a kiválasztott rekordot.

A route tehát tulajdonképpen egy **összekötő pont** a böngésző kérése és a PHP kód logikája között. Ha nincs megfelelő route, a Laravel nem tudja, melyik programrésznek kellene lefutnia.

12.2. A controller mint vezérlőközpont

A projekt legfontosabb vezérlője a ProductController, mert a webshop fő funkciói itt találhatók meg. Ez a controller kezeli a terméklista betöltését, az új termék mentését, a képek megjelenítését, a részletes nézetet, a törlést és a leírás frissítését.

Például a listaoldal működését ez a rövid metódus mutatja:

```
public function index()
{
    $data = Product::latest()->paginate();
    return view('welcome', compact('data'));
}
```

Ennek a működése a következő:

1. a Product::latest() lekéri a termékeket,
2. a paginate() lapozható formában adja vissza őket,
3. a view('welcome', compact('data')) átadja az adatokat a nézetnek.

A részletes oldal ennél is egyszerűbb logikát követ:

```
public function show_details($id)
{
    $data = Product::where('id', '=', $id)->latest()->paginate(1);
    return view('show_details', compact('data'));
}
```

Itt a controller csak az adott azonosítójú terméket kéri le, majd átadja a show_details nézetnek.

A controller tehát nem maga jeleníti meg az oldalt, és nem is ő maga az adatbázis. Az ő feladata az, hogy **összehangolja** a többi részt.

12.3. A modell és az adatbázis kapcsolata

A Laravelben a modell egy adatbázistábla objektum-orientált megfelelője. A projektben a Product modell a products táblához kapcsolódik. Ez a modell mondja meg, hogy a termék milyen mezőkkel rendelkezik.

```
class Product extends Model
{
    protected $table = 'products';

    protected $fillable = [
        'pro_name_EN',
        'pro_description_EN',
        'pro_price',
        'pro_image',
    ];
}
```

Ez azt jelenti, hogy a rendszer a terméknél a következő adatokat kezeli:

- angol név,
- angol leírás,
- ár,

- kép.

Az adatbázis tábla létrehozását a migráció végzi. A projekt migrációja megmutatja, milyen mezők jönnek létre a products táblában.

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->String('pro_name_EN');
    $table->Float('pro_price');
    $table->timestamps();
});
```

A további mezők külön SQL utasítással kerülnek hozzáadásra:

```
DB::statement("ALTER TABLE products ADD pro_image LONGBLOB");
DB::statement("ALTER TABLE products ADD pro_description_EN
MEDIUMTEXT");
```

Ez szakmailag azért érdekes, mert a kép nem fájlként, hanem **bináris adatként** kerül az adatbázisba a LONGBLOB mezőben. A leírás pedig hosszabb szövegként MEDIUMTEXT típusban tárolódik.

12.4. Az új termék létrehozásának folyamata

Az új termék felvitele a new_product.blade.php oldalon indul. Ez a nézet tartalmazza az adatbeviteli űrlapot, amely POST kéréssel elküldi az adatokat a newproduct/store útvonalra. Az enctype="multipart/form-data" azért szükséges, mert képfeltöltés is történik.

```
<form method="post" action="{{ url('newproduct/store') }}"
enctype="multipart/form-data">
    @csrf
```

A @csrf védelem a Laravel biztonsági mechanizmusa, amely megakadályozza az illetéktelen űrlapküldéseket.

A leírás mező például így néz ki:

```
<textarea
    id="productdescription_en"
    class="form-control"
    name="productdescription_en"
    rows="6"
    required
></textarea>
```

A mentés gomb szintén jól mutatja, hogy ez az oldal a Create műveletet valósítja meg:

```
<button type="submit" class="luxury-btn">
    <i class="fas fa-save"></i>
    Save Product
</button>
```

A nézet ráadásul a hibákat és a sikeres mentés üzenetét is kiírja:

```
@if($errors->any())
@if(session()->has('success'))
```

Ez azért jó megoldás, mert a felhasználó azonnal visszajelzést kap arról, hogy a mentés sikeres volt-e.

12.5. Validáció és mentés a controllerben

Az űrlap elküldése után a store() metódus fut le. Ez a metódus először ellenőrzi, hogy a beküldött adatok megfelelnek-e az elvárt formátumnak.

```
$request->validate([
    'product_name_en' => 'required|string|max:255',
    'productdescription_en' => 'required|string',
    'Product_Price' => 'required|numeric',
```

```
        'image' => 'required|image',
    ]);
```

Ez soronként a következőt jelenti:

- a product_name_en kötelező és szöveges adat,
- a leírás kötelező,
- az árnak számnak kell lennie,
- a feltöltött fájlnak valódi képnek kell lennie.

A kép feldolgozása az Intervention Image csomag segítségével történik:

```
$image = Image::make($request->file('image'))->encode('jpeg');
```

Ez a sor beolvassa a feltöltött képet, majd JPEG formátumba alakítja. Ezután a program létrehoz egy új terméket:

```
$product = new Product();
$product->pro_name_EN = $request->product_name_en;
$product->pro_description_EN = $request->productdescription_en;
$product->pro_price = $request->Product_Price;
$product->pro_image = (string) $image;
$product->save();
```

A save() a legfontosabb pont, mert itt lesz a PHP objektumból tényleges adatbázis-rekord. A mentés után a rendszer visszairányít, és sikerüzenetet ad:

```
return redirect()->back()->with('success', $product->pro_name_EN . '
added successfully to the database');
```

Ez a teljes adatmentési folyamat egyik legtisztább példája a projektben.

12.6. A képek speciális kezelése

A projekt egyik technikailag legérdekesebb része, hogy a képek nem egy storage vagy public mappában vannak, hanem közvetlenül az adatbázisban.

Emiatt a böngésző nem egy hagyományos fájlútvonalról kapja meg a képet, hanem egy külön route segítségével.

A kép route-ja:

```
Route::get('/w/show/{id}', [ProductController::class,'show']);
```

A show() metódus így működik:

```
$image = Product::findOrFail($id);
$image_file = Image::make($image->pro_image);
$response = Response::make($image_file->encode('jpeg'));
$response->header('Content-Type', 'image/jpeg');
return $response;
```

Ez a folyamat egyszerű nyelven:

1. megkeresi a terméket az adatbázisban,
2. kiolvassa a képet a pro_image mezőből,
3. képként visszaküldi a böngészőnek.

A nézet oldalán emiatt elég ennyi:

```
<img src="/w/show/{{ $row->id }}?size=pdhi" class="product-image"
alt="{{ $row->pro_name_EN }}">
```

A Blade sablon tehát nem tudja, hol van fizikailag a kép, csak annyit tud, hogy van egy URL, amely képként szolgálja ki azt. Ez jó példa a rétegek szétválasztására.

12.7. A listaoldal dinamikus működése

A welcome.blade.php fájl a főoldal megjelenítéséért felel. Itt történik a termékek ciklikus kirajzolása. Laravel Blade-ben ez a @foreach szerkezettel valósul meg.

```
@foreach($data as $row)
```

A terméknév kiírása például így történik:

```
{{ $row->pro_name_EN }}
```

A részletező oldalra mutató link pedig így épül fel:

```
<a href="/show_details/{{ $row->id }}">
```

A főoldalon tehát minden kártya ugyanabból a logikából épül fel:

- kép,
- név,
- részletek link.

A menüben a Products legördülő lista is dinamikusan a termékekből épül fel:

```
@foreach($data as $row)
    <li>
        <a href="/show_details/{{ $row->id }}" class="dropdown-
item">
            {{ $row->pro_name_EN }}
        </a>
    </li>
@endforeach
```

Ez azt mutatja, hogy ugyanaz az adatforrás több felületi elemet is kiszolgálhat.

12.8. A részletező oldal és az adatmódosítás

A show_details.blade.php nézet nemcsak megjeleníti a termék teljes adatlapját, hanem bizonyos adminisztratív műveleteket is biztosít. A termék neve, ára, leírása és képe egy oldalon jelenik meg.

A törléshez tartozó űrlap:

```
<form action="{{ route('products.destroy', $row->id) }}"
      method="POST"
      onsubmit="return confirm('Are you sure you want to delete this
product?')">
    @csrf
    @method('DELETE')
```

Ez azért fontos, mert a HTML önmagában nem tud DELETE kérést küldeni, ezért a Laravel a @method('DELETE') segítségével „utánozza” ezt a HTTP műveletet.

A leírás frissítéséhez külön űrlap tartozik:

```
<form action="{{ route('products.updateDescription', $row->id) }}"
method="POST">
    @csrf
    <textarea name="pro_description_EN" class="luxury-textarea">{{
$row->pro_description_EN }}</textarea>
```

A gomb pedig elküldi az új értéket:

```
<button type="submit" class="luxury-btn luxury-btn-primary">
    <i class="fa fa-save mr-2"></i>Update Description
</button>
```

A controllerben ezt a következő metódus kezeli:

```
public function updateDescription(Request $request, $id)
{
    $product = Product::findOrFail($id);
    $product->pro_description_EN = $request->pro_description_EN;
    $product->save();

    return redirect()->back()->with('success', 'Description updated
```

```
successfully');
}
```

Ez egy nagyon jól érthető példa arra, hogyan történik egy meglévő rekord részleges módosítása.

12.9. Bejelentkezés, felhasználókezelés és menüsor

A projekt használja a Laravel beépített hitelesítési rendszerét. Ezt már a route-fájl elején jelzi az alábbi sor:

```
Auth::routes();
```

Ez automatikusan létrehozza a login, logout és egyéb auth útvonalakat.

A főoldali Blade sablon külön kezeli a vendég és a bejelentkezett felhasználó nézetét:

```
@guest
    <a class="nav-link" href="{{ route('login') }}">{{ __('Login')
}}</a>
@endguest
```

Bejelentkezett felhasználónál pedig a név és a kijelentkezés jelenik meg:

```
{{ Auth::user()->name }}
<form id="logout-form" action="{{ route('logout') }}" method="POST">
    @csrf
</form>
```

Ez azt mutatja, hogy a menürendszer már fel van készítve a felhasználói állapot kezelésére.

A HomeController külön auth middleware-t használ:

```
public function __construct()
{
```

```
        $this->middleware('auth');
    }
```

Ez azt jelenti, hogy elvileg a hozzá tartozó oldalak csak belépés után lennének elérhetők. Ugyanakkor a jelenlegi route-fájl alapján a /home útvonal ténylegesen már a ProductController@index metódusra mutat, tehát a HomeController inkább egy korábbi auth-alapú szerkezet része lehet. Ez szakmailag fontos megfigyelés, mert mutatja, hogy a projekt fejlődött, és bizonyos régebbi elemek megmaradtak benne.

12.10. Szakmai megjegyzések és továbbfejlesztési pontok

A projekt működőképes, de a kódban látszanak olyan részek is, amelyek még fejleszthetők. Az egyik ilyen az általános szerkesztési logika, mert az edit() és update() metódusok jelenleg üresek:

```
    public function edit($id)
    {
        //
    }

    public function update(Request $request, $id)
    {
        //
    }
```

Ez azt jelenti, hogy a teljes körű termékszerkesztés még nincs kész, csak a leírás módosítása működik külön metódussal.

A keresési funkcióban is látható egy technikai probléma. A keresés ilyen mezőnevekkel dolgozik:

```
Product::where('pro_name', 'LIKE', '%'.$search_val.'%')
        ->orwhere('pro_description', 'LikE', '%'.$search_val.'%')
        ->orwhere('pro_Price', 'LIKE', '%'.$search_val.'%')
```

Miközben a tényleges adatbázismezők ezek:

```
'pro_name_EN',
'pro_description_EN',
'pro_price',
```

Vagyis a keresés valószínűleg egy korábbi adatstruktúrához készült, és jelen állapotában javításra szorul. Ez a dolgozatban külön jó szakmai észrevétel lehet, mert azt mutatja, hogy nemcsak leírod a rendszert, hanem érted is a működését.

Ugyancsak fontos megjegyezni, hogy az admin és az új termék felvitelére szolgáló útvonalak jelenleg publikus route-ként szerepelnek:

```
Route::get('/admin', [ProductController::class, 'admin']);
Route::get('/newproduct', [ProductController::class, 'create'])-
>name('newproduct');
Route::post('/newproduct/store', [ProductController::class,
'store'])->name('newproduct.store');
```

Ezeket egy további fejlesztési lépésben célszerű lenne auth middleware-rel védeni, hogy csak bejelentkezett vagy admin jogosultságú felhasználó érhesse el őket.

12.11. Összefoglaló technikai értelmezés

A rendszer működésének lényege tehát az, hogy a böngészőben végzett művelet egy route-hoz kerül, a route meghívja a megfelelő controller metódust, a controller a Product modellen keresztül adatot olvas vagy ír az adatbázisba, majd az eredményt Blade sablonon keresztül jeleníti meg. A termékek listázása, a részletes oldal, a mentés, a törlés és a

leírásfrissítés mind ezt az elvet követik. A képek különösen jó technikai példát mutatnak arra, hogy nemcsak egyszerű szöveges adatok, hanem bináris tartalmak is kezelhetők Laravelben és MySQL-ben.

13. Továbbfejlesztési lehetőségek

A fejlesztett webshop jelenlegi formájában már alkalmas a termékek megjelenítésére, feltöltésére, részletes megtekintésére, valamint bizonyos adatok módosítására és törlésére. Ugyanakkor a projekt szerkezete és kialakítása alapján jól látható, hogy a rendszer további fejlesztésekre is alkalmas, így a későbbiekben egy még összetettebb és felhasználóbarátabb webalkalmazássá alakítható.

A továbbfejlesztés egyik legfontosabb iránya a navigációs sávban szereplő menüpontok teljes funkcionalitásának kidolgozása. A Products menü tovább bővíthető lenne kategóriák szerinti szűréssel, keresési lehetőséggel és rendezési funkciókkal, ami megkönnyítené a termékek közötti eligazodást. Az About Us oldal részletesebb bemutatkozó tartalmat tartalmazhatna, ahol a webshop célja, háttere és küldetése is megjelenhetne. A Contact Us menüpont fejlesztésével kapcsolatfelvételi űrlap, elérhetőségi adatok vagy akár automatikus üzenetküldési lehetőség is kialakítható lenne.

Szintén fontos fejlesztési lehetőség a többnyelvű működés teljes megvalósítása. A navigációban megjelenő Language menüpont arra utal, hogy a rendszer a jövőben több nyelven is használhatóvá válhat. Ez nemcsak a felhasználói élményt javítaná, hanem lehetőséget adna arra is, hogy a webshop szélesebb közönség számára legyen elérhető. Egy magyar és angol nyelvű felület már önmagában is jelentős előrelépést jelentene a rendszer professzionalizálása szempontjából.

A bejelentkezési rendszer további bővítése szintén lényeges fejlesztési irány lehet. A jövőben elkülöníthetők lennének a normál felhasználói és adminisztrátori jogosultságok, így a rendszer biztonságosabban és szabályozottabban kezelhetné az egyes műveleteket. Ennek részeként

kialakítható lenne regisztráció, személyes felhasználói profil, rendelési előzmények megtekintése, valamint különböző jogosultsági szintek bevezetése is.

Összességében elmondható, hogy a projekt nemcsak egy működő vizsgafeladatként értelmezhető, hanem egy olyan fejlesztési alapként is, amelyre a későbbiekben egy összetettebb, modernebb és valós környezetben is jól használható webshop építhető. A jelenlegi rendszer stabil kiindulópontot biztosít ahhoz, hogy a későbbi bővítések során a funkcionalitás, a felhasználói élmény és a biztonság egyaránt tovább növekedjen.

14. Tesztelés, dokumentálás és beadási szempontok

Egy Laravel projekt elkészítése önmagában még nem jelenti azt, hogy a rendszer szakmailag is teljesnek tekinthető. A beadás minőségét jelentősen javítja, ha a fejlesztő röviden bemutatja, hogyan történt a tesztelés, milyen fő funkciókat próbált ki, és milyen eredményeket kapott.

Tesztelés során érdemes ellenőrizni, hogy a termékek listázása megfelelően működik-e, az űrlapok validációja kiszűri-e a hibás adatokat, a képfeltöltés elfogadja-e a megfelelő fájlokat, valamint a törlés és módosítás valóban frissíti-e az adatbázist. Ugyanilyen fontos annak bemutatása is, hogy a rendszer hibás bemenet esetén nem omlik össze, hanem kulturált visszajelzést ad a felhasználónak.

A beadandó dokumentációban célszerű külön jelezni a használt route-okat, a fő controller metódusokat, a nézetek szerepét és az adatbázis mezőit. Ezzel a dokumentum nemcsak leíró jellegű lesz, hanem a fejlesztési munka valódi szakmai hátterét is bemutatja.

6. táblázat: Ellenőrzés

Ellenőrzési terület	Mit érdemes tesztelni?	Elvárt eredmény
Terméklista	Betöltődnek-e az adatok az adatbázisból?	A termékek hibamentesen megjelennek.
Űrlapvalidáció	Kötelező mezők és numerikus ár ellenőrzése	Hibás adat esetén hibaüzenet jelenik meg.
Képfeltöltés	Csak képfájl fogadható-e el?	A rendszer csak megfelelő fájltípust ment.
Módosítás és törlés	Frissül-e az adatbázis tartalma?	A változások azonnal láthatók a felületen.

15. Összegzés

Összességében elmondható, hogy a Laravel egy korszerű és jól használható PHP keretrendszer, amely kiváló alapot biztosít adatbázis-kapcsolt webalkalmazások fejlesztéséhez. A HTML, PHP, CSS és MySQL technológiák együtt alkotják azt a technikai hátteret, amelyre egy működő webshop épülhet.

A keretrendszer legnagyobb előnye, hogy a fejlesztést rendezett szerkezetben támogatja. Ez különösen fontos beadandó projektek esetén, mert a kész rendszer nemcsak működőképes, hanem szakmailag is jól bemutatható lesz.

16. Felhasznált irodalom

[1] Laravel. Laravel 8.x Documentation. Hivatalos dokumentáció.
https://laravel.com/docs/8.x

[2] Laravel. Middleware | Laravel 8.x. Hivatalos dokumentáció.
https://laravel.com/docs/8.x/middleware

[3] Laravel. Starter Kits | Laravel 8.x. Hivatalos dokumentáció.
https://laravel.com/docs/8.x/starter-kits

[4] Laravel. Database: Migrations. Hivatalos dokumentáció.
https://laravel.com/docs/13.x/migrations

[5] PHP Documentation Group. PHP Manual – Dealing with Forms.
https://www.php.net/manual/en/tutorial.forms.php

[6] PHP Documentation Group. PHP Manual – Handling File Uploads.
https://www.php.net/manual/en/features.file-upload.php

[7] MDN Web Docs. <form>: The Form element.
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/form

[8] MDN Web Docs. CSS: Cascading Style Sheets.
https://developer.mozilla.org/en-US/docs/Web/CSS

[9] MySQL. MySQL 8.4 Reference Manual.
https://dev.mysql.com/doc/en/

[10] MySQL. CREATE TABLE Statement.
https://dev.mysql.com/doc/en/create-table.html

[11] MySQL. Data Types.
https://dev.mysql.com/doc/en/data-types.html

[12] Bootstrap. Bootstrap 4.6 Documentation.
https://getbootstrap.com/docs/4.6/getting-started/introduction/